

A close-up photograph of pink cherry blossoms on dark, thin branches. The flowers are in various stages of bloom, with some showing prominent stamens. The background is a soft, out-of-focus white and light pink, suggesting a bright, sunny day. The overall mood is fresh and delicate.

Fundamentals of Machine Learning

Fundamentals of Machine Learning

- Machine learning $\neq$ just training a model
- Goal: generalization, not training performance
- Decisions before training matter

This Chapter Builds

- How to evaluate models correctly
- How data & features shape learning
- Why models overfit
- A universal ML workflow

Forms of Machine Learning

Machine learning problems can be grouped into 4 main branches

1. Supervised Learning
2. Unsupervised Learning
3. Self-supervised Learning
4. Reinforcement Learning

Supervised Learning (การเรียนรู้แบบมีผู้สอน)

Supervised Learning

Learning a mapping from **inputs** to **known targets**

- Training data has **labels / targets**
- Goal: learn **the relationship between $x \rightarrow y$**

Supervised Learning

Typical supervised problems:

- Binary classification
- Multiclass classification
- Regression

Examples:

- Image classification
- Speech recognition
- Language translation

Supervised Learning – รูปแบบที่ซับซ้อนขึ้น

More advanced supervised tasks:

- Sequence generation
- Syntax tree prediction
- Object detection
- Image segmentation

Unsupervised Learning

Unsupervised Learning

Learning patterns from data **without targets**

- No labels
- Focus on data structure
- Used for understanding data

Unsupervised Learning

Used for:

- Data visualization
- Data compression
- Denoising
- Understanding correlations

Common categories:

- Clustering
- Dimensionality reduction

Self-supervised Learning

Self-supervised Learning

Supervised learning **without human-annotated labels**

- Still has targets
- Targets are **generated from input data**

Self-supervised Learning

Examples:

- Autoencoders (input = target)
- Predict next frame in video
- Predict next word in text

Reinforcement Learning

Reinforcement Learning

Learning by interacting with an environment to maximize reward

- Agent
- Environment
- Actions
- Reward

Reinforcement Learning

- Applications:
- Games (Atari, Go)
- Robotics
- Self-driving cars
- Resource management

Evaluating machine-learning models

Evaluating Machine-Learning Models

- Goal: **generalization**
- Main problem: **overfitting**
- Need reliable **evaluation**

Overfitting vs Generalization

- Training performance → always improves
- Performance on new data → may stall or worsen
- This is overfitting

Core Idea of Evaluation

Key principle

You can only control what you can observe

- Must **measure generalization**
- Evaluation is essential

Three Data Sets (Big Picture)

Data Splitting

- Training set
- Validation set
- Test set

Training Set

Training set

- Used to **train the model**
- Used to update **parameters (weights)**

Validation Set

Validation set

- Used to **evaluate during development**
- Used to tune **hyperparameters**

Why Not Just Train + Test?

Why not only two sets?

- Train on training set
- Evaluate on test set

test set มาช่วยตัดสินใจ แปลว่า test set ไม่บริสุทธิ์อีกต่อไป

Information Leak

Information leak

- Validation data leaks into model
- Happens during hyperparameter tuning

Role of Test Set

Test set

- Used **once**
- Never used for tuning
- Final measure of generalization

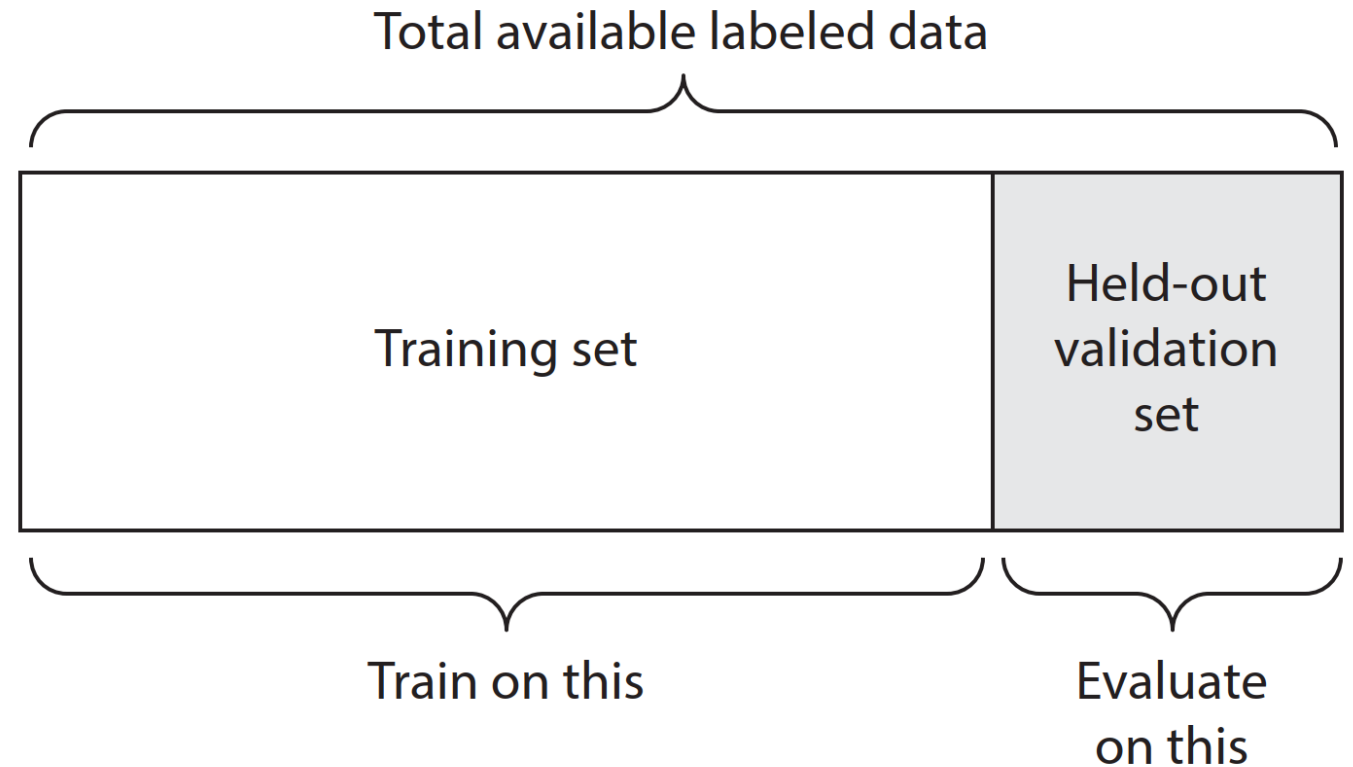
Simple Hold-out Validation

Hold-out validation

- Split data once
- Train / Validation / Test

Limitation of Hold-out

- Small validation/test sets
- High variance of results



Simple Hold-out Validation

```
num_validation_samples = 10000
np.random.shuffle(data)
validation_data = data[:num_validation_samples]
data = data[num_validation_samples:]
training_data = data[:]
model = get_model()
model.train(training_data)
validation_score = model.evaluate(validation_data)
# At this point you can tune your model,
# retrain it, evaluate it, tune it again...
model = get_model()
model.train(np.concatenate([training_data, validation_data]))
test_score = model.evaluate(test_data)
```

K-Fold Validation

K-fold validation

- Split data into K folds
- Train on K-1 folds
- Validate on 1 fold
- Average the scores

Why K-Fold Helps

- Reduces variance
- More reliable estimate
- Still needs test set

K-Fold Validation

```
k = 4
num_validation_samples = len(data) // k
np.random.shuffle(data)
validation_scores = [ ]
for fold in range(k):
    validation_data = data[ num_validation_samples * fold:num_validation_samples * (fold + 1) ]
    training_data = ( data[:num_validation_samples * fold] + data[num_validation_samples * (fold + 1):] )

    model = get_model()
    model.train(training_data)
    validation_score = model.evaluate(validation_data)
    validation_scores.append(validation_score)

validation_score = np.average(validation_scores)

model = get_model()
model.train(data)
test_score = model.evaluate(test_data)
```

Iterated K-Fold with Shuffling

Iterated K-fold

- Repeat K-fold multiple times
- Shuffle data each time
- Average all scores

Things to Keep in Mind

Choosing evaluation protocol

- Data representativeness
- Arrow of time
- Redundancy in data

Data Representativeness

- Training & test must represent data
- Always shuffle (most cases)

Arrow of Time

- Time-series data
- Do not shuffle
- Prevent temporal leak

Redundancy in Data

- Duplicate or similar samples
- Training and validation must be disjoint

Data Preprocessing, Feature Engineering, and Feature Learning

Why Data Preparation Matters

Before building a model

- How should we prepare input data and targets before feeding them into a neural network?

Three Related Concepts

Key concepts

- Data preprocessing
- Feature engineering
- Feature learning

Data Preprocessing

Data preprocessing

- Making raw data more amenable to neural networks

Includes:

- Vectorization
- Normalization
- Handling missing values
- Feature extraction

Vectorization

Vectorization

- Converting data into tensors
- Inputs and targets must be tensors
- Usually floating-point tensors

Vectorization

Examples:

- Text → list of integers → one-hot encoding → float32 tensor
- Digits / house prices → already vectorized

Value Normalization

Value normalization

- Making feature values small and homogeneous
- Typical range: 0–1
- Similar scale across features

Normalization

Digit classification:

- Pixel values: 0–255
- Convert to float32
- Divide by 255 $\rightarrow$ 0–1

Normalization

House-price prediction:

- Features with different ranges
- Normalize each feature independently
- Mean = 0, Std = 1

Why Normalization Matters

- Good input data should:
- Take small values
- Be homogeneous

Handling Missing Values

Missing values

When some feature values are unavailable

Example:

- House-price data
- Crime rate missing for some samples

Handling Missing Values

General rule:

- Input missing values as 0
- If 0 is not a meaningful value

Missing Values

Problem:

- Train data has no missing values
- Test data has missing values

Solution:

- Artificially generate missing values during training

Feature Engineering

Feature engineering

Applying non-learned transformations to make the problem easier

- Uses:
- Domain knowledge
- Human understanding of the problem

Clock Example: Raw Pixels

- ใช้ pixel ดิบ
- ต้องใช้ CNN
- ใช้ทรัพยากรสูง

Raw data:
pixel grid



- คึงพิกัด (x, y) ของเข็ม
- โมเดลง่ายลงมาก

Better
features:
clock hands'
coordinates

{x1: 0.7,
y1: 0.7}
{x2: 0.5,
y2: 0.0}

{x1: 0.0,
y1: 1.0}
{x2: -0.38,
y2: 0.32}

- แปลงเป็น polar coordinates
- ใช้มุม θ ของเข็ม
- แทนไม่ต้องใช้ ML

Even better
features:
angles of
clock hands

theta1: 45
theta2: 0

theta1: 90
theta2: 140

Core idea: Make the problem simpler by expressing it better

Feature Engineering (Before Deep Learning)

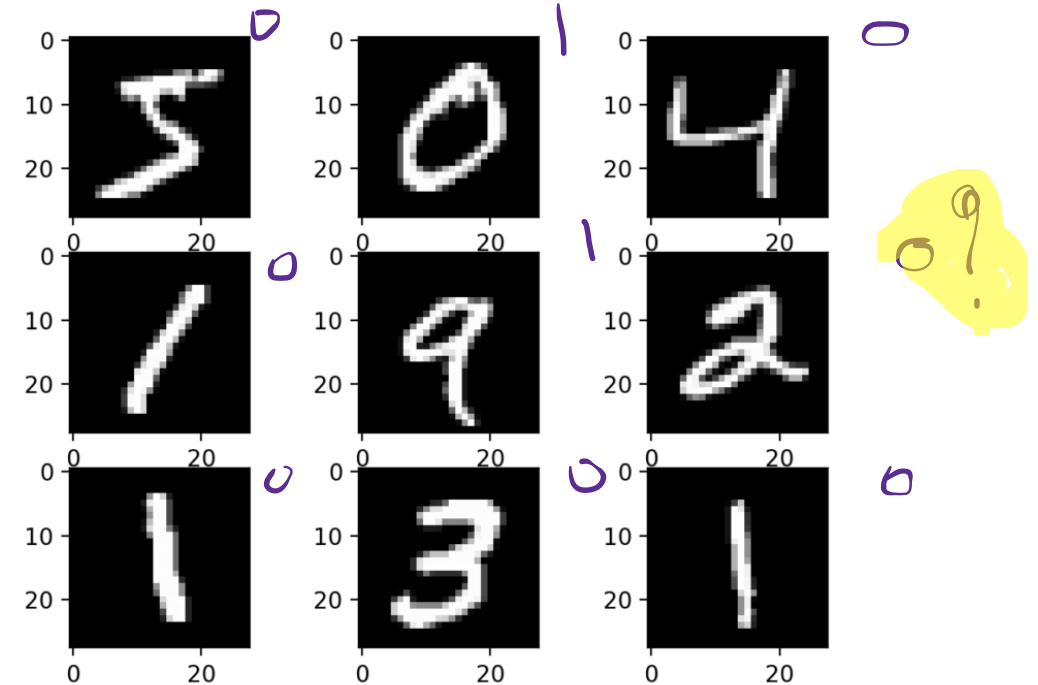
Before deep learning:

- Shallow models
- Needed handcrafted features

Examples:

- Loops in digits
- Digit height
- Pixel histograms

Loops in digits



Feature Learning (Deep Learning)

Feature learning

- Neural networks automatically learn features from raw data

Do We Still Need Feature Engineering?

Ans: Yes

Reasons:

- Fewer resources
- Less data required

Overfitting and underfitting

Overfitting และ Underfitting คืออะไร และทำไมสำคัญ

Overfitting & Underfitting

- ปัญหาหลักของ **machine learning**
- เกิดในแทบทุกโมเดล
- เกี่ยวข้องโดยตรงกับ **generalization**

นิยาม Overfitting และ Underfitting

Definitions

- **Overfitting**

โมเดลเรียนรู้ *training data* ดีเกินไป แต่ทำงานแย่บน *test data*

- **Underfitting**

โมเดลง่ายเกินไปยังจับ **pattern** สำคัญไม่ได้

Optimization vs Generalization

- **Optimization**

ทำให้โมเดลดีที่สุดบน *training data*

- **Generalization**

ทำให้โมเดลดีบน *unseen data (test data)*

- เราควบคุม optimization ได้

แต่ ควบคุม **generalization** โดยตรงไม่ได้

พฤติกรรมของโมเดลระหว่างการฝึก

Training dynamics

- ช่วงต้น:
 - training loss ↓
 - validation loss ↓ → **underfitting**
- หลังจากนั้น:
 - training loss ↓
 - validation loss ↑ → **overfitting**

ทางแก้ Overfitting และแนวคิด Regularization

How to fight overfitting

- วิธีที่ดีที่สุด: **more training data** → **overfitting** ↓
- ถ้าทำไม่ได้ → **regularization**
- **Regularization**
= การจำกัดความสามารถในการจดจำของโมเดล

Model Capacity (แนวคิดสำคัญมาก)

Model Capacity

- จำนวน **learnable parameters** ขึ้นกับ:
 - จำนวน layers
 - จำนวน units ต่อ layer

capacity สูง $\rightarrow$ memorization สูง

Original model

```
from keras import models
from keras import layers
```

```
model = models.Sequential()
model.add(layers.Dense(16, activation='relu',
input_shape=(10000,)))
model.add(layers.Dense(16, activation='relu'))
model.add(layers.Dense(1, activation='sigmoid'))
```

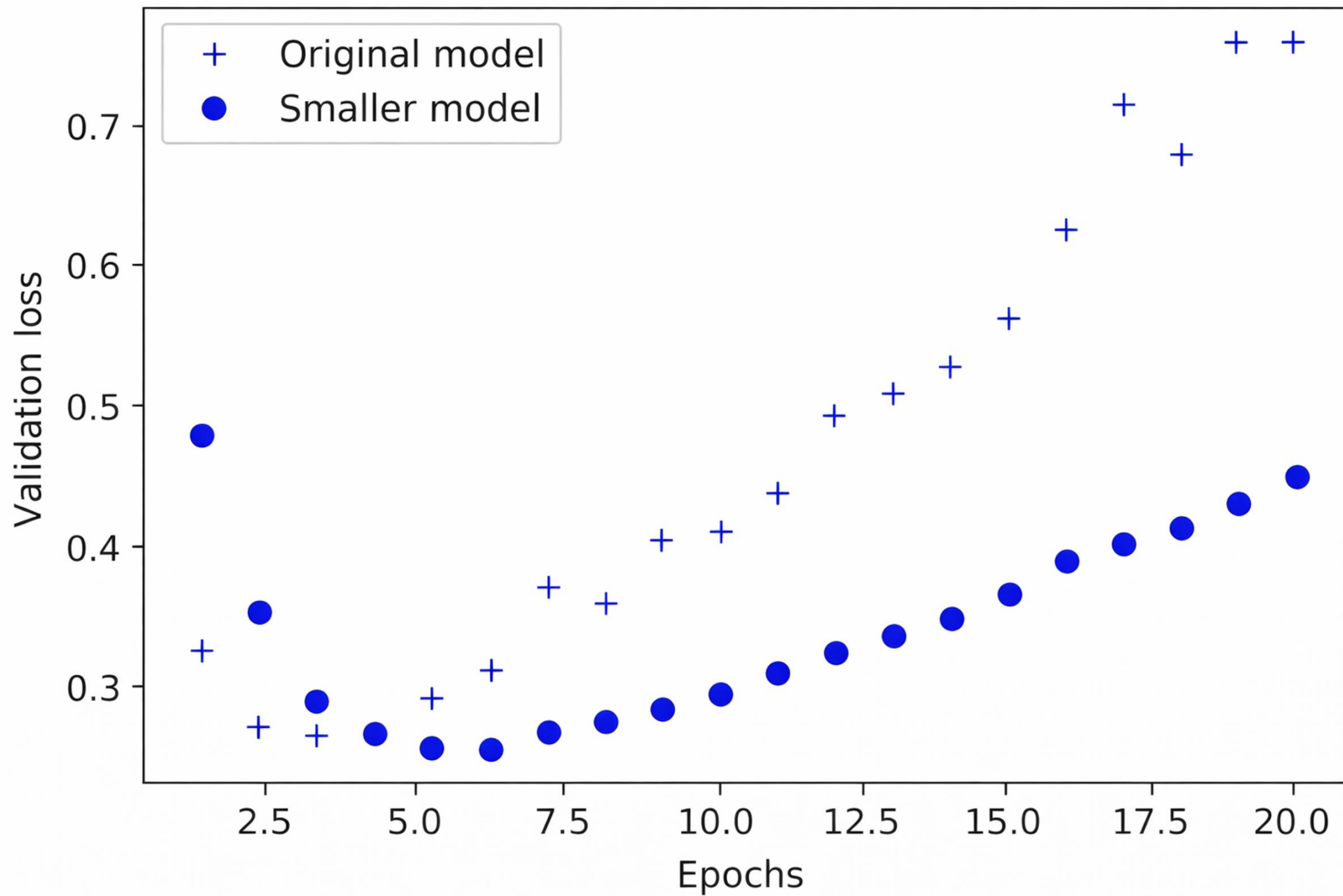
Reducing Network Size (Regularization แบบง่ายสุด)

Reduce model capacity

- ลดจำนวน units
- ลดจำนวน layers
- หา **sweet spot** บน validation set

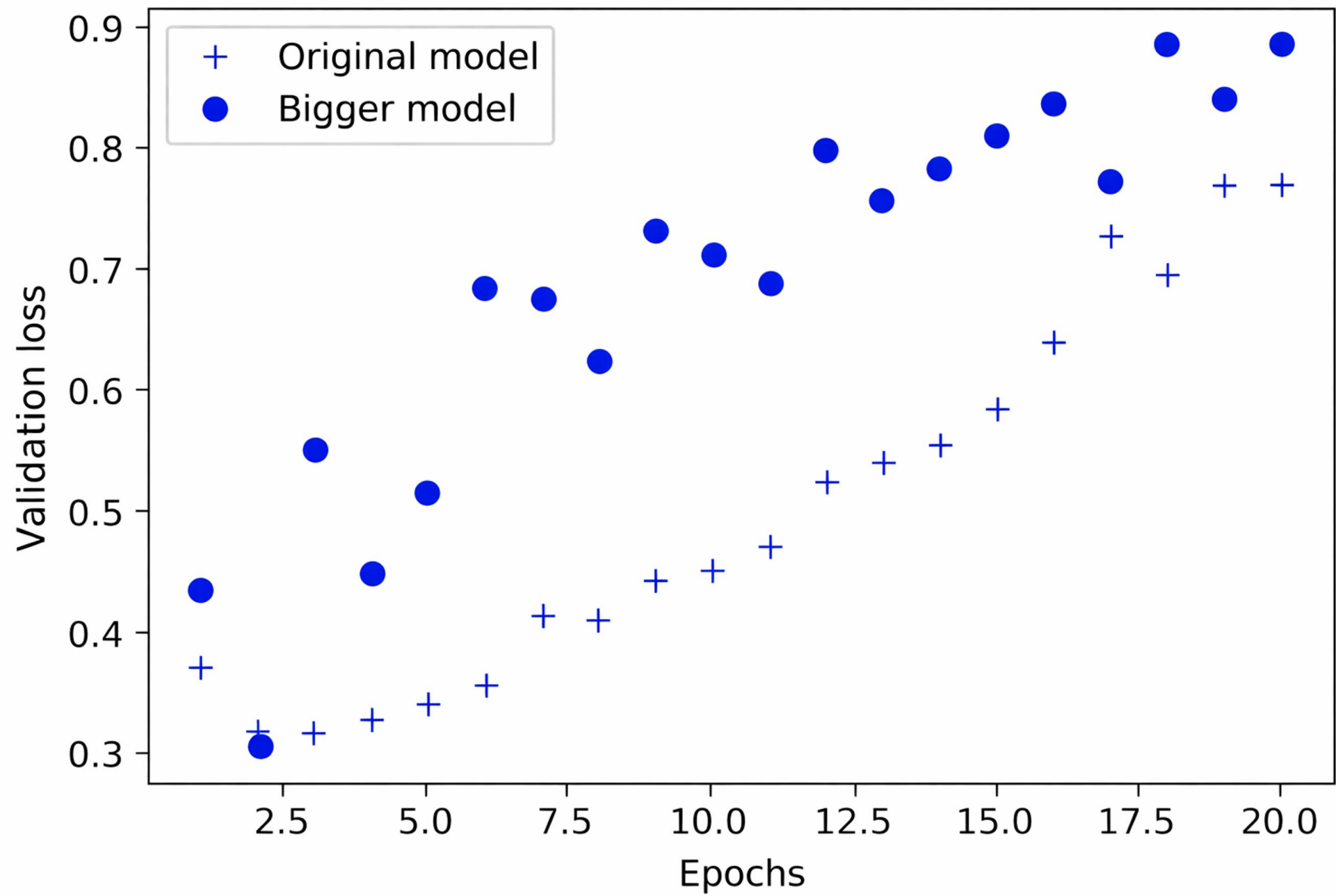
Lower Capacity Mode

```
model = models.Sequential()  
model.add(layers.Dense(4, activation='relu',  
input_shape=(10000,)))  
model.add(layers.Dense(4, activation='relu'))  
model.add(layers.Dense(1, activation='sigmoid'))
```



Higher Capacity Model

```
model = models.Sequential()  
model.add(layers.Dense(512, activation='relu',  
input_shape=(10000,)))  
model.add(layers.Dense(512, activation='relu'))  
model.add(layers.Dense(1, activation='sigmoid'))
```



Comparing the three models

Model	Capacity	Overfitting	Validation loss	Epochs
Smaller model				
Original model				
Larger model				

Why the original model is the best choice

Original model ไม่ใช่โมเดลที่มี **capacity** สูงที่สุด แม้ว่าโมเดลอาจเริ่มเกิด **overfitting** เร็วกว่าบางโมเดล แต่โมเดลนี้สามารถให้ผลลัพธ์ดังต่อไปนี้:

- **validation loss** ที่ต่ำกว่า
- ใช้จำนวน **epochs** น้อยกว่า

ดังนั้นจึงหมายความว่า:

- โมเดลมี **model complexity** ต่ำกว่าเท่าที่จำเป็นมี **generalization-complexity trade-off** ที่ดีกว่า

สรุปคือ

Best model $\neq$ highest capacity model

Weight Regularization (L1 / L2)

Weight Regularization

เพิ่ม **penalty** ให้ weight ที่มีค่ามาก

- **L1 regularization** $\rightarrow |weight|$
- **L2 regularization (weight decay)** $\rightarrow weight^2$

แนวคิดจาก **Occam's razor**

$$Loss_{L_1} = Loss + \lambda \sum_{i=1}^m |w_i|$$

Adding L2 Weight Regularization

```
from keras import regularizers
```

```
model = models.Sequential()  
model.add(layers.Dense(16, kernel_regularizer=regularizers.l2(0.001),  
activation='relu', input_shape=(10000,)))  
model.add(layers.Dense(16, kernel_regularizer=regularizers.l2(0.001),  
activation='relu'))  
model.add(layers.Dense(1, activation='sigmoid'))
```

```
from keras import regularizers  
regularizers.l1(0.001)  
regularizers.l1_l2(l1=0.001, l2=0.001)
```

Dropout

Dropout

- เทคนิค **regularization** ที่ใช้บ่อยมากใน neural networks
- พัฒนาโดย **Geoff Hinton**
- ใช้เพื่อลด **overfitting**

หลักการทำงานของ Dropout

- สุ่มปิด (drop out) neuron บางตัว
- ตั้งค่า output ของ neuron เหล่านั้นเป็น 0
- ทำแบบสุ่มทุก batch / ทุก iteration

[0.2, 0.5, 1.3, 0.8, 1.1]



[0, 0.5, 1.3, 0, 1.1]

Dropout rate และพฤติกรรมช่วง test

Dropout rate

- สัดส่วนของ neuron ที่ถูกปิดและ neuron ทั้งหมด
- ค่าที่ใช้บ่อย: 0.2-0.5

ในขั้นตอน test

- ไม่มี neuron ใดถูกปิด
- output ถูก rescale

Dropout (concept)

At train time:

```
layer_output *= np.random.randint(0, high=2, size=layer_output.shape)
```

At test time:

```
layer_output *= 0.5
```

At train time:

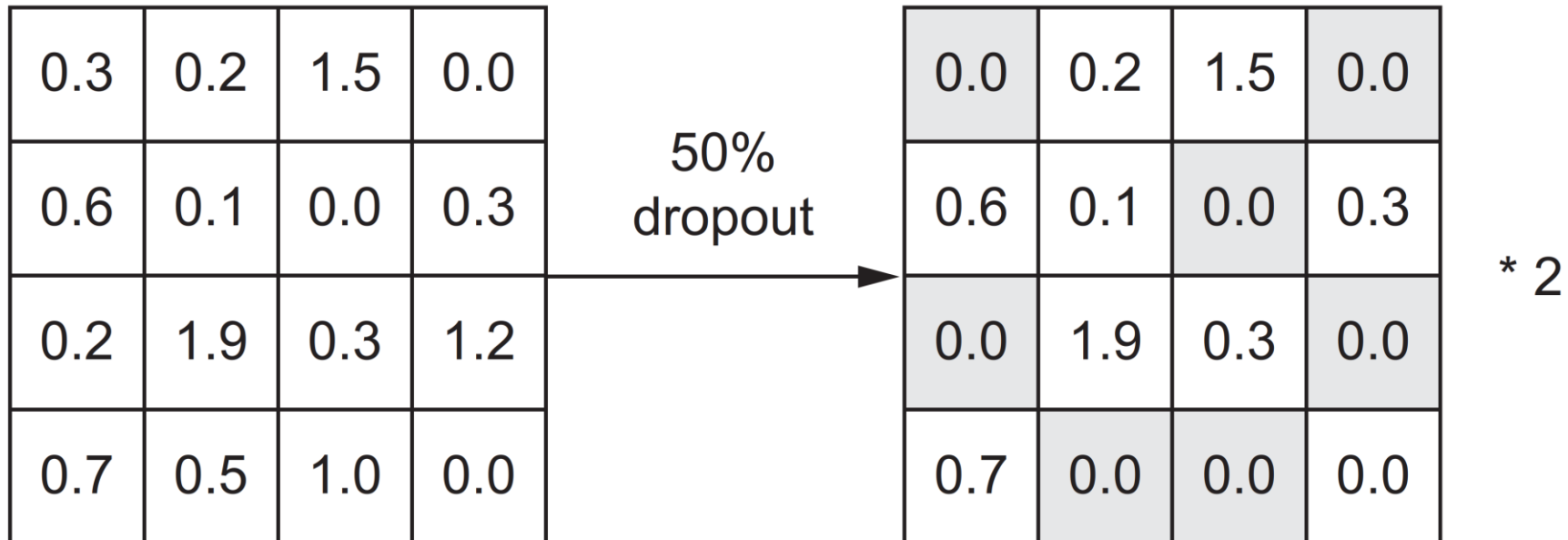
```
layer_output *= np.random.randint(0, high=2, size=layer_output.shape)
```

```
layer_output /= 0.5
```

```
layer_output *= random_mask
```

```
layer_output /= (1 - dropout_rate)
```

At test time: -



Dropout ใน Keras

```
model = models.Sequential()  
model.add(layers.Dense(16, activation='relu', input_shape=(10000,)))  
model.add(layers.Dropout(0.5))  
model.add(layers.Dense(16, activation='relu'))  
model.add(layers.Dropout(0.5))  
model.add(layers.Dense(1, activation='sigmoid'))
```

